

NAME: MICHAEL OCHIENG'

REG NO: CIT-227-033/2024

COURSE: SOFTWARE ENGINEERING

UNIT: FOUNDATIONS OF AI

QUESTION ONE (15 Marks)

(a) The Concept of Search: Informed vs. Uninformed Search

Search in Artificial Intelligence refers to the systematic process by which an agent explores a problem space — a set of possible states and transitions — to find a sequence of actions that leads from an initial state to a goal state. Search algorithms are foundational to AI problem solving, planning, and decision-making.

Uninformed Search (Blind Search)

Uninformed search strategies operate using only the information provided in the problem definition: the initial state, the set of possible actions, and the goal test. They have no additional knowledge about which direction is most promising, so they explore states systematically without guidance.

- Breadth-First Search (BFS): Expands all nodes at the current depth before moving deeper. It is complete and optimal for uniform step costs, but requires significant memory.
- Depth-First Search (DFS): Expands the deepest unexplored node first. It is memory-efficient but can get stuck in infinite paths and is not guaranteed to find the optimal solution.
- Uniform Cost Search (UCS): Always expands the node with the lowest cumulative path cost. It is optimal when step costs are variable.
- Iterative Deepening Search (IDDFS): Combines BFS completeness with DFS memory efficiency by running depth-limited searches with increasing depth bounds.

Informed Search (Heuristic Search)

Informed search strategies use a heuristic function $h(n)$, which estimates the cost from a given node to the goal. This problem-specific knowledge guides the search toward more promising areas of the problem space, making it significantly more efficient.

- Greedy Best-First Search: Expands the node that appears closest to the goal based on $h(n)$ alone. It is fast but not always optimal.
- A* Search: Uses the evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the start to node n , and $h(n)$ is the estimated cost to the goal. A* is both complete and optimal, provided $h(n)$ is admissible (never overestimates).
- Hill Climbing and Beam Search are other informed techniques that trade completeness for speed.

Key Differences between Informed and Uninformed Search:

The primary distinction is the use of a heuristic. Uninformed search treats all unexplored nodes equally and expands them based solely on structural properties (depth, cost). Informed search uses domain knowledge via $h(n)$ to prioritize nodes that are more likely to lead to the goal quickly. As a result, informed search is generally more efficient (fewer nodes expanded), though it requires a well-designed heuristic. Uninformed search is more general — it works

on any problem without additional knowledge — but tends to be slower and more memory-intensive.

(b) Python Implementation of A* Search Strategy

The following Python program implements the A* Search algorithm to find the optimal (least-cost) path in a weighted graph. The algorithm uses the evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is the actual path cost from the start node and $h(n)$ is an admissible heuristic estimate to the goal.

```
import heapq

# -----
# A* Search Algorithm Implementation
# f(n) = g(n) + h(n)
#   g(n) = actual cost from start to node n
#   h(n) = heuristic estimate from node n to goal
# -----

def astar_search(graph, heuristics, start, goal):
    """
    Performs A* Search on a weighted graph.
    Parameters:
        graph      : dict {node: [(neighbor, cost), ...]}
        heuristics : dict {node: estimated_cost_to_goal}
        start      : starting node
        goal       : goal node
    Returns: (path, total_cost) or (None, infinity)
    """
    # Priority queue stores (f_cost, g_cost, node, path_so_far)
    open_list = []
    heapq.heappush(open_list, (heuristics[start], 0, start, [start]))

    # Track best known g-cost for each visited node
    visited = {}

    while open_list:
        f_cost, g_cost, current, path = heapq.heappop(open_list)

        # Skip if this node was already visited with a lower cost
        if current in visited and visited[current] <= g_cost:
            continue
        visited[current] = g_cost

        print(f' Node: {current}  g={g_cost}  h={heuristics[current]}
              f={f_cost}')

        # Check if we reached the goal
        if current == goal:
```

```

        return path, g_cost

    # Expand each neighbor of the current node
    for neighbor, step_cost in graph.get(current, []):
        new_g = g_cost + step_cost
        new_f = new_g + heuristics.get(neighbor, 0)
        heapq.heappush(open_list, (new_f, new_g, neighbor, path +
[neighbor]))

    return None, float('inf') # No path found

# --- Example: Graph representing cities with travel costs ---
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('C', 2), ('D', 5)],
    'C': [('D', 1), ('E', 3)],
    'D': [('E', 1)],
    'E': []
}

# Heuristic: estimated straight-line distance to goal 'E'
heuristics = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 2,
    'E': 0 # Goal node always has h = 0
}

print('A* Search Traversal:')
path, cost = astar_search(graph, heuristics, start='A', goal='E')

print()
if path:
    print(f'Optimal Path: {" -> ".join(path)}')
    print(f'Total Cost : {cost}')
else:
    print('No path found.')

```

Expected Output:

```

A* Search Traversal:
Node: A  g=0  h=7  f=7
Node: B  g=1  h=6  f=7
Node: C  g=3  h=4  f=7
Node: D  g=4  h=2  f=6
Node: E  g=5  h=0  f=5

Optimal Path: A -> B -> C -> D -> E
Total Cost : 5

```

The algorithm guarantees an optimal path because the heuristic $h(n)$ is admissible — it never overestimates the true cost to the goal. The min-heap ensures that the node with the lowest $f(n)$ is always expanded next, minimizing the total path cost.

(c)(i) Logical Concepts: Valid, Satisfiable, Unsatisfiable, and Falsifiable

Valid (Tautology): A formula is valid if it evaluates to TRUE under every possible interpretation of its propositional variables. A valid formula is true regardless of the truth values assigned to its components. Example: $P \vee \neg P$ is always true.

Satisfiable: A formula is satisfiable if there exists at least one interpretation under which it evaluates to TRUE. It does not have to be true in all cases — just in at least one. Example: $P \wedge Q$ is satisfiable because it is true when both P and Q are true.

Unsatisfiable (Contradiction): A formula is unsatisfiable if it evaluates to FALSE under every possible interpretation. There is no assignment of truth values that makes it true. Example: $P \wedge \neg P$ is always false — a contradiction.

Falsifiable: A formula is falsifiable if there exists at least one interpretation under which it evaluates to FALSE. Note that every formula that is not a tautology is falsifiable. Example: $P \wedge Q$ is falsifiable because it is false when P is false.

Relationship summary: A valid formula is necessarily satisfiable (true everywhere implies true somewhere), but a satisfiable formula is not necessarily valid. An unsatisfiable formula is not satisfiable and not falsifiable in the usual sense — it is always false. Every non-tautology is falsifiable.

(c)(ii) Truth Table Analysis for $P \wedge Q \rightarrow \neg R$

The formula to evaluate is: $P \wedge Q \rightarrow \neg R$. This reads as: 'If P and Q are both true, then R must be false.' The implication $A \rightarrow B$ is false only when A is true and B is false. Below is the complete truth table for all 8 possible combinations of P, Q, and R:

P	Q	R	$P \wedge Q$	$\neg R$	$P \wedge Q \rightarrow \neg R$
T	T	T	T	F	F
T	T	F	T	T	T
T	F	T	F	F	T
T	F	F	F	T	T
F	T	T	F	F	T
F	T	F	F	T	T
F	F	T	F	F	T
F	F	F	F	T	T

Classification and Analysis:

Valid? NO. The formula is not valid because it is false in row 1 ($P=T, Q=T, R=T$). A valid formula must be true in all rows.

Satisfiable? YES. The formula is true in 7 out of 8 rows (e.g., row 2 where $P=T$, $Q=T$, $R=F$ gives TRUE). At least one satisfying interpretation exists.

Unsatisfiable? NO. The formula is not false in all rows — it has many true interpretations.

Falsifiable? YES. The formula is false in row 1 ($P=T$, $Q=T$, $R=T$), so at least one falsifying interpretation exists.

Condition for TRUE: The formula holds true whenever P and Q are NOT both true simultaneously, OR when R is FALSE. In other words, the formula fails only when the antecedent ($P \wedge Q$) is satisfied while the consequent ($\neg R$) is not.

Condition for FALSE: The formula evaluates to FALSE ONLY under the single condition: $P = \text{TRUE}$, $Q = \text{TRUE}$, and $R = \text{TRUE}$ at the same time. This is the only case where the implication fails.

QUESTION TWO (10 Marks)

(a) PEAS and PAGE Analysis: Automatic DNA & Genomic Sequencing System

The Shakahola Automatic DNA and Genomic Sequencing System is an AI agent designed to match DNA profiles extracted from unidentified victims with samples provided by relatives, enabling rapid and accurate identification of the deceased.

PEAS Framework:

Performance Measure (P): The agent is evaluated based on the accuracy of DNA matches (percentage of correctly identified victims), the speed of processing each sample, the false positive and false negative rates in match results, the total number of victims successfully identified within a given timeframe, and compliance with forensic and legal standards.

Environment (E): The operational environment includes biological samples (tissue, bone, hair) collected from Shakahola mass grave sites, reference DNA samples voluntarily provided by relatives of missing persons, forensic laboratory equipment (centrifuges, PCR machines, gel electrophoresis systems), high-throughput DNA sequencing machines, national and international DNA databases, and the human forensic team overseeing the process.

Actuators (A): The agent acts on the environment through the following outputs: generating match reports linking a victim's profile to a relative's sample, updating the centralized victims identification database, flagging potential matches above a confidence threshold for human expert review, producing genealogical relationship trees, and sending automated notifications to the relevant government agencies.

Sensors (S): The agent perceives its environment through DNA sequencers that read raw genomic data from samples, barcode and RFID scanners that track physical sample integrity and chain of custody, database query interfaces that retrieve existing profiles for comparison, quality control sensors that flag degraded or contaminated samples, and digital input forms where relatives submit their personal and biological information.

PAGE Framework:

Partially Observable (P): The environment is only partially observable. The agent cannot guarantee access to complete information — some relatives may not come forward to provide samples, samples from badly decomposed bodies may be incomplete or contaminated, and the full scope of victims may not be known at any given time.

Multi-Agent (A): The system operates as a multi-agent system. Multiple specialized sub-agents work cooperatively: a sample intake agent that logs and tracks incoming specimens, a sequencing agent that processes raw genetic data, a matching agent that performs probabilistic comparisons, a reporting agent that compiles and communicates findings, and a database management agent that maintains up-to-date records.

Stochastic (G): The environment is stochastic rather than deterministic. DNA matching produces probabilistic confidence scores rather than absolute certainties. Sample degradation, contamination, partial sequences, and the absence of close relatives all introduce uncertainty that the agent must handle through probabilistic reasoning and thresholding.

Sequential (E): The task is sequential, not episodic. Each identification decision affects subsequent ones — updating the database changes what future matches are possible, and a confirmed identification removes a victim's profile from the unmatched pool. Long-term consistency and accuracy depend on the cumulative state of the system.

Additional environment properties: the environment is Dynamic (new samples arrive continuously), Continuous (DNA sequences are high-dimensional continuous data rather than discrete symbols), and the task is carried out in a setting where ethical and legal constraints are significant performance considerations.

(b) Vacuum Cleaner Agent — Design and Implementation

The following program implements a simple reflex-based vacuum cleaner agent operating in a multi-room grid environment. Each room in the grid can be in one of two states: Dirty or Clean. The agent perceives its current room's status and decides whether to clean it or move toward the nearest dirty room.

```
import random

# =====
# VACUUM CLEANER AGENT
# Environment : A 3x3 grid of rooms, each 'Dirty' or 'Clean'
# Percept      : Status of the current room
# Actions      : Suck (clean current room), Move (navigate grid)
# Goal         : Ensure all rooms are clean
# =====

class VacuumEnvironment:
    """Represents the grid environment with multiple rooms."""

    def __init__(self, rows=3, cols=3):
        self.rows = rows
        self.cols = cols
        # Randomly assign each room as Dirty or Clean at startup
        self.grid = {
            (r, c): random.choice(['Dirty', 'Clean'])
            for r in range(rows) for c in range(cols)
        }
        self.agent_pos = (0, 0) # Agent starts at top-left corner
        self.total_cleans = 0
        self.total_moves = 0

    def display(self):
        """Print the current grid state. Agent position shown in
        brackets."""
        print(' Environment:')
        for r in range(self.rows):
            row_str = ' '
            for c in range(self.cols):
```



```

        cell = 'D' if self.grid[(r, c)] == 'Dirty' else 'C'
        if (r, c) == self.agent_pos:
            row_str += f'[{cell}] '
        else:
            row_str += f' {cell} '
        print(row_str)

def is_all_clean(self):
    """Return True if every room in the grid is clean."""
    return all(status == 'Clean' for status in self.grid.values())

class VacuumAgent:
    """
    Simple Reflex Vacuum Agent.
    Rule 1: If current room is Dirty -> SUCK
    Rule 2: If current room is Clean -> MOVE toward nearest dirty room
    """

    def __init__(self, environment):
        self.env = environment

    def perceive(self):
        """Read the status of the current room."""
        return self.env.grid[self.env.agent_pos]

    def act(self):
        """Execute one action based on current perception. Returns False
when done."""
        if self.env.is_all_clean():
            print(' All rooms clean. Task complete.')
            return False

        status = self.perceive()
        pos = self.env.agent_pos

        if status == 'Dirty':
            # SUCK: Clean the current room
            self.env.grid[pos] = 'Clean'
            self.env.total_cleans += 1
            print(f' Action: SUCK at {pos} -> Room cleaned.')
        else:
            # MOVE: Navigate one step toward the nearest dirty room
            target = self.find_nearest_dirty()
            if target:
                self.env.agent_pos = self.step_toward(pos, target)
                self.env.total_moves += 1
                print(f' Action: MOVE {pos} -> {self.env.agent_pos}')
        return True

```

```

def find_nearest_dirty(self):
    """Scan grid row by row and return the first dirty room found."""
    for r in range(self.env.rows):
        for c in range(self.env.cols):
            if self.env.grid[r, c] == 'Dirty':
                return (r, c)
    return None

def step_toward(self, current, target):
    """Move one step closer to target: vertical movement first, then
horizontal."""
    cr, cc = current
    tr, tc = target
    if cr < tr: return (cr + 1, cc) # Move Down
    elif cr > tr: return (cr - 1, cc) # Move Up
    elif cc < tc: return (cr, cc + 1) # Move Right
    elif cc > tc: return (cr, cc - 1) # Move Left
    return current

# --- Main Simulation ---
def run_simulation():
    print('=== VACUUM CLEANER AGENT SIMULATION ===')
    env = VacuumEnvironment(rows=3, cols=3)
    agent = VacuumAgent(env)

    print('Initial state:')
    env.display()

    step = 0
    max_steps = 60 # Safety limit to prevent infinite loops

    while step < max_steps:
        step += 1
        print(f'\nStep {step}:')
        if not agent.act():
            break
        env.display()

    print(f'\nSimulation ended after {step} steps.')
    print(f'Rooms cleaned : {env.total_cleans}')
    print(f'Moves made      : {env.total_moves}')

run_simulation()

```

Design Notes:

- The environment is a 3x3 grid with rooms randomly initialized as Dirty or Clean at startup.

- The agent follows two simple reflex rules: if the current room is dirty, clean it (SUCK); otherwise, move toward the nearest dirty room.
- `find_nearest_dirty()` performs a row-by-row scan and returns the coordinates of the first dirty room found.
- `step_toward()` implements single-step navigation: the agent first resolves vertical distance, then horizontal, ensuring it always moves in a direct path.
- The `max_steps` guard prevents the simulation from running indefinitely in edge cases.
- The agent terminates automatically when all rooms are clean, then prints a performance summary showing total cleans and moves.
- The agent is a simple reflex agent — it acts based purely on the current percept with no memory of past states.

— *End of Answers* —